

Youtube2knowledge

Whitepaper tecnico del codigo Python

Del agente de conocimiento local al servicio web verificable

*Analisis detallado de cada modulo, funcion e instruccion Python
que compone el pipeline de extraccion, sintesis y consulta de conocimiento*

Infraestructura NVIDIA DGX Spark · Qwen2.5 · Nemotron · FastAPI

2 de septiembre de 2026

Índice general

1. Introduccion y alcance	2
2. El modelo de datos: <code>schemas.py</code>	3
2.1. Analisis instruccion por instruccion	3
3. Normalizacion de URLs: <code>normalize.py</code>	5
3.1. Analisis	5
4. Persistencia SQLite: <code>storage.py</code> e <code>ingest.py</code>	6
4.1. Analisis instruccion por instruccion	6
5. Extraccion visual: <code>vision_extract.py</code>	8
5.1. Analisis del flujo	8
6. Verificacion visual: <code>vision_verify.py</code>	9
6.1. Intencion del esquema	9
7. Busqueda hibrida: <code>hybrid_search.py</code>	10
7.1. La formula de combinacion	10
7.2. Analisis de la formula	10
8. El backend FastAPI: <code>api_v08.py</code>	11
8.1. El protocolo asincrono <code>/ask</code>	11
8.2. Manejo defensivo de excepciones	11
8.3. El peligro de los <code>f-string</code> anidados	12
9. Estados persistentes: <code>pipeline_state.py</code>	13
9.1. Intencion de la maquina de estados	13
10. Archivado idempotente: <code>drive_archive.py</code>	14
11. Helper de inferencia: <code>ollama_pdf_researcher.py</code>	15
11.1. Analisis clave por clave	15
12. La capa determinista: sanitizacion de sintesis	16
12.1. Analisis	16
12.2. Extraccion de tokens con Python inline	16
13. Sintesis: los principios de ingenieria del codigo Python	18

Capítulo 1

Introduccion y alcance

Este whitepaper documenta, con detalle instruccion por instruccion, todos los fragmentos de codigo **Python** que aparecen en la construccion del sistema que evoluciono de un agente de conocimiento local (**knowledge-agent**) hacia la aplicacion web **Youtube2knowledge**. El objetivo no es solo mostrar el codigo, sino explicar *la intencion* detras de cada decision: por que la instruccion existe, que problema resuelve y que principio de ingenieria encarna.

El sistema completo se apoya en una arquitectura que ejecuta toda la inferencia de inteligencia artificial de forma local sobre una NVIDIA DGX Spark, empleando Parakeet/Qwen para percepcion, Qwen2.5-VL para vision, Nemotron para sintesis, vLLM y Ollama para inferencia, SQLite con FTS5 para persistencia y busqueda, FastAPI como backend, Next.js en Vercel como frontend, y Cloudflare Tunnel para exponer el servicio local a Internet.

Intencion del codigo

El hilo conductor de todo el codigo Python del proyecto es un mismo principio: *colocar una capa determinista entre un modelo probabilistico y la persistencia*. Python no se usa aqui como pegamento superficial, sino como la capa que impone contratos, valida salidas y garantiza que un LLM nunca escriba directamente en la base de conocimiento sin verificacion previa.

Los modulos Python analizados, agrupados por responsabilidad, son:

Modulo	Responsabilidad
schemas.py	Definicion de estructuras de datos canonicas.
normalize.py	Limpieza y normalizacion de URLs y dominios.
storage.py / ingest.py	Persistencia SQLite sin alterar el texto original.
vision_extract.py	Extraccion visual con Qwen2.5-VL via Ollama.
vision_verify.py	Segunda pasada de verificacion visual con JSON Schema.
hybrid_search.py	Busqueda hibrida lexical + semantica.
api_v08.py	Backend FastAPI: pipeline, estados, endpoints.
pipeline_state.py	Maquina de estados persistente y reanudable.
drive_archive.py	Archivado idempotente en Google Drive.
ollama_pdf_researcher.py	Helper de inferencia Nemotron via Ollama.
Sanitizador de sintesis	Capa determinista de validacion de contratos.

Capítulo 2

El modelo de datos: `schemas.py`

El primer código Python del proyecto define *que* es una unidad de conocimiento. Antes de extraer, sintetizar o buscar, el sistema necesita una forma canónica de representar una entrada. Para ello se emplean `dataclasses`, que ofrecen tipado explícito y valores por defecto seguros.

```
1 from dataclasses import dataclass, field
2 from typing import List, Optional
3
4 @dataclass
5 class Resource:
6     url: str
7     domain: str = ""
8     resource_type: str = ""
9     title: str = ""
10
11 @dataclass
12 class KnowledgeEntry:
13     user_note: str = ""
14     urls: List[str] = field(default_factory=list)
15     images: List[str] = field(default_factory=list)
16
17     title: str = ""
18     user_topics: List[str] = field(default_factory=list)
19     detected_topics: List[str] = field(default_factory=list)
20     people: List[str] = field(default_factory=list)
21     organizations: List[str] = field(default_factory=list)
22     concepts: List[str] = field(default_factory=list)
23     resources: List[Resource] = field(default_factory=list)
24     summary: str = ""
25
26     source: str = "manual"
27     timestamp: Optional[str] = None
```

2.1. Análisis instruccional por instrucción

`from dataclasses import dataclass, field`. Importa el decorador `@dataclass`, que genera automáticamente `__init__`, `__repr__` y `__eq__`, y la función `field`, necesaria para declarar valores por defecto mutables de manera segura.

`from typing import List, Optional`. Aporta anotaciones de tipo. `Optional[str]` equivale a `str | None`: el campo puede contener una cadena o estar ausente. Estas anotaciones son documentación ejecutable y habilitan verificación estática.

@dataclass class Resource. Modela un recurso externo (un enlace) como una entidad de primera clase con `url`, `domain`, `resource_type` y `title`. Solo `url` es obligatorio; los demás se completan durante la normalización.

`urls: List[str] = field(default_factory=list)`. Esta es la instrucción más importante del bloque. En Python, escribir `urls: List[str] = []` sería un error grave: la lista se compartiría entre todas las instancias de la clase. `field(default_factory=list)` construye una *lista nueva* en cada instanciación, evitando el clásico bug de argumentos por defecto mutables.

Intención del código

La separación entre `user_topics` y `detected_topics`, o entre lo que aporta el usuario y lo que infiere el modelo, es deliberada. El diseño distingue desde el tipo de dato *la procedencia* de cada campo. El sistema nunca confunde una afirmación humana con una inferencia automática: esa distinción es la base del *grounding* verificable que caracteriza al proyecto.

El campo `source = "manual"` etiqueta el origen (manual, vision, PDF, YouTube), y `timestamp` permite ordenar y deduplicar temporalmente.

Capítulo 3

Normalizacion de URLs: `normalize.py`

Las entradas del mundo real llegan sucias. Un usuario puede pegar un enlace en formato Markdown, con parentesis duplicados o con la URL repetida. El modulo `normalize.py` limpia URLs, convierte enlaces Markdown a URL real, extrae dominios y deduplica.

La prueba del modulo se ejecuto con un *heredoc* de Python:

```
1 python3 - <<'PY'
2 from app.normalize import normalize_url, get_domain
3
4 urls = [
5     "[https://developer.nvidia.com/](https://developer.nvidia.com/)",
6     "[https://arxiv.org/](https://arxiv.org/)"
7 ]
8
9 for url in urls:
10     clean = normalize_url(url)
11     print(clean, "->", get_domain(clean))
12 PY
```

3.1. Analisis

`python3 - «'PY' ... PY`. El guion (-) indica a Python que lea el programa desde la entrada estandar. El *heredoc* entrega el codigo entre los marcadores. Al entrecomillar 'PY', el shell no expande variables dentro del bloque, garantizando que el codigo llega literal al interprete.

Las entradas de prueba son deliberadamente patologicas: URLs envueltas en sintaxis Markdown [texto](enlace) con la direccion duplicada. Este es el caso exacto que rompio la primera version del sistema.

`normalize_url(url)` extrae la URL real desde la envoltura Markdown, y `get_domain(clean)` aisla el dominio. El resultado correcto fue `developer.nvidia.com` y `arxiv.org`.

Principio de diseno

El normalizador es determinista, no un modelo. Extraer un dominio de una URL es una operacion que *no debe* delegarse a un LLM: es una tarea con respuesta unica y verificable. El proyecto reserva la IA para lo ambiguo y usa codigo Python clasico para lo que tiene una respuesta correcta.

Capítulo 4

Persistencia SQLite: `storage.py` e `ingest.py`

El modulo `storage.py` crea la base `knowledge.db` y la tabla `entries`. El modulo `ingest.py` normaliza y guarda una entrada *sin permitir que el modelo cambie el texto original*. La verificación de persistencia se realizó con Python directo sobre SQLite:

```
1 import sqlite3
2 from pathlib import Path
3
4 db = Path.home() / "knowledge-agent" / "data" / "knowledge.db"
5 conn = sqlite3.connect(db)
6
7 rows = conn.execute(
8     "SELECT id, source, user_note, urls FROM entries"
9 ).fetchall()
10
11 print("ENTRIES:", len(rows))
12 for row in rows:
13     print(row)
14
15 conn.close()
```

4.1. Analisis instruccion por instruccion

`import sqlite3`. SQLite viene incluido en la biblioteca estandar de Python. No requiere servidor ni dependencias externas, lo que lo hace ideal para un sistema *on-premise* de un solo nodo.

`from pathlib import Path`. `pathlib` maneja rutas de forma orientada a objetos y portable entre sistemas operativos, superior a concatenar cadenas manualmente.

`db = Path.home() / "knowledge-agent" / "data" / "knowledge.db"`. El operador `/` sobrecargado en `Path` construye la ruta al archivo de base de datos partiendo del directorio del usuario. Esto evita rutas absolutas fragiles.

`conn = sqlite3.connect(db)`. Abre (o crea) la conexión a la base.

`conn.execute("SELECT ...").fetchall()`. Ejecuta la consulta y recupera todas las filas. `fetchall()` materializa el cursor en una lista de tuplas.

`print(.ENTRIES:", len(rows))`. Verifica el conteo, una invariante clave: tras cada mutación importante el proyecto comprueba conteos.

`conn.close()`. Cierra la conexión liberando el bloqueo del archivo, importante porque SQLite bloquea a nivel de archivo.

Intencion del codigo

La regla central de `ingest.py` es que el texto original del usuario es *immutable*. El modelo puede anadir metadatos (temas, organizaciones, resumen) pero jamas reescribir la nota original. Esto preserva la fidelidad de la fuente, requisito indispensable de un sistema de conocimiento verificable.

Capítulo 5

Extraccion visual: vision_extract.py

Este script convierte una imagen (una diapositiva, una captura) en JSON estructurado usando el modelo multimodal `qwen2.5v1:7b` servido por Ollama. Su flujo logico es:

1. leer la imagen;
2. convertirla a Base64;
3. llamar a Ollama en `127.0.0.1:11434`;
4. usar `qwen2.5v1:7b`;
5. exigir salida JSON;
6. guardar un archivo `*.visual.json`.

5.1. Analisis del flujo

Leer la imagen y convertirla a Base64. Las APIs de modelos multimodales transportan imagenes como texto codificado. Base64 traduce bytes binarios a caracteres ASCII seguros para incrustar en un cuerpo JSON.

Llamar a Ollama en 127.0.0.1:11434. La direccion de bucle local (*loopback*) es intencional: el modelo nunca se expone a la red. Toda la inferencia permanece dentro del Spark.

Exigir JSON. Aqui aparecio el primer fallo real del sistema:

```
1 JSONDecodeError: Unterminated string
```

En una imagen densa, el modelo produjo una cadena JSON truncada.

Principio de diseno

El prompt engineering ayuda, pero un pipeline robusto debe imponer la estructura de salida mediante JSON Schema cuando el *runtime* lo permite. Pedir "devuelve JSON" no es un contrato; declarar un esquema formal si lo es.

Capítulo 6

Verificacion visual: `vision_verify.py`

La leccion anterior condujo a una segunda etapa. `vision_verify.py` entrega de nuevo la imagen original junto con la primera extraccion, y Qwen verifica especificamente ortografia de nombres, organizaciones, proyectos, acronimos, numeros, monedas, unidades, URLs visibles y campos inciertos.

La salida se *obliga* mediante un JSON Schema con esta forma:

```
1 {
2   "source_type": "",
3   "title": "",
4   "people": [],
5   "organizations": [],
6   "projects": [],
7   "detected_topics": [],
8   "concepts": [],
9   "metrics": [],
10  "links_visible": [],
11  "links_mentioned": [],
12  "summary": "",
13  "verification_notes": [],
14  "uncertain_fields": []
15 }
```

6.1. Intencion del esquema

Cada clave del esquema resuelve un problema concreto de extraccion visual:

- `links_visible` frente a `links_mentioned` separa un enlace que aparece escrito en pantalla de uno solo aludido verbalmente. Sin esta distincion el sistema podria inventar URLs.
- `verification_notes` registra la justificacion de la segunda pasada.
- `uncertain_fields` es una senal de humildad epistemica: el modelo declara *lo que no esta seguro de haber leído bien*, permitiendo al pipeline tratar esos campos con cautela.

Intencion del codigo

Este esquema materializa el principio de *doble verificacion visual*. La primera pasada extrae; la segunda comprueba contra la imagen original. El coste de una segunda inferencia se justifica porque los errores de OCR en nombres propios, cifras y acronimos son precisamente los mas costosos de detectar despues.

Capítulo 7

Busqueda hibrida: `hybrid_search.py`

El sistema combina dos senales de recuperacion: coincidencia lexical exacta (SQLite FTS5) y similitud semantica (embeddings `nomic-embed-text`). El endpoint `/search` genera el embedding de la consulta en tiempo real, recupera coincidencias lexicales, calcula similitud coseno contra los embeddings almacenados y combina ambas senales.

7.1. La formula de combinacion

La ponderacion hibrida usada es:

$$S_{hybrid} = 0,35 S_{lexical} + 0,65 S_{semantic}^{norm}$$

con la normalizacion del coseno al rango $[0, 1]$:

$$S_{semantic}^{norm} = \frac{S_{cosine} + 1}{2}.$$

7.2. Analisis de la formula

0.65 para lo semantico, 0.35 para lo lexical. La ponderacion favorece la semantica sin eliminar la evidencia lexical. En investigacion, una consulta suele expresar una idea con vocabulario distinto al de la fuente, por lo que la semantica captura sinonimos y parafrasis. Pero los nombres propios, proyectos y terminos tecnicos siguen beneficiandose de la coincidencia exacta de FTS5.

$(S_{cosine} + 1) / 2$. La similitud coseno vive en $[-1, 1]$. Sumar 1 y dividir entre 2 la reescala a $[0, 1]$, haciendola comparable con el puntaje lexical antes de combinar. Sin esta normalizacion, una correlacion negativa podria restar de forma incoherente.

Principio de diseno

La deduplicacion acompaña a la busqueda: una firma formada por titulo, organizaciones y proyectos normalizados evita que replicas de regresion dominen el ranking. El sistema conserva historicos y pruebas sin mostrar copias equivalentes al usuario.

Capítulo 8

El backend FastAPI: `api_v08.py`

El corazon del servicio es `api_v08.py`, ejecutado de forma persistente mediante `systemd` con `Uvicorn`. Contiene los endpoints, el pipeline y la orquestacion.

8.1. El protocolo asincrono `/ask`

La version sincrona hacia todo en una sola peticion HTTP larga:

HTTP request → retrieval → LLM → HTTP response

La version asincrona separa aceptacion y computo:

POST `/ask` → 202 + token, GET `/ask-status/token` → estado o resultado

Intencion del codigo

El protocolo de dos fases (token + sondeo) evita depender de una conexion HTTP prolongada. Un LLM local puede tardar decenas de segundos; mantener abierta la conexion durante todo ese tiempo es fragil ante timeouts y reconexiones. Devolver un token de inmediato y consultar el estado despues hace el sistema robusto ante errores temporales de red, exactamente la propiedad que el frontend mejorado (commit 50f6315) explota para reanudar el trabajo tras una recarga.

8.2. Manejo defensivo de excepciones

El endpoint envuelve el trabajo pesado en un bloque de captura:

```
1 try:
2     ...
3 except Exception as exc:
4     mark_failed(...)
5     raise HTTPException(...)
```

`except Exception as exc` captura cualquier fallo del pipeline. `mark_failed(...)` persiste el error en el estado del paquete *antes* de propagarlo. `raise HTTPException(...)` traduce el fallo interno a una respuesta HTTP deliberada.

Principio de diseno

Este patron tuvo una consecuencia sutil: al convertir la excepcion en una `HTTPException`, Uvicorn dejo de imprimir el traceback completo. La leccion fue que la observabilidad no debe depender de logs; el estado persistente por token (`current_stage`, `failed_stage`, `error`) se convirtio en la fuente autoritativa de diagnostico.

8.3. El peligro de los f-string anidados

Al generar codigo o JSON desde Python surgio un error de sintaxis. Dentro de un `f-string`, las llaves tienen significado especial:

```
1 # Interpretado como expresion interpolada VACIA (error):
2 json.loads(package_final or "{}")
3
4 # Correccion: llaves escapadas para producir literalmente {}
5 json.loads(package_final or "{{}}")
```

Analisis. En un `f-string`, `{}` intenta interpolar una expresion. Para emitir las llaves literales `{}` hay que duplicarlas: `{{}}`. El fallo se validaba antes de reiniciar el servicio con:

```
1 python3 -m py_compile api_v08.py
```

`python3 -m py_compile` compila el archivo a bytecode sin ejecutarlo, detectando errores de sintaxis de forma barata.

Principio de diseno

El codigo generado debe validarse sintacticamente antes de entrar al servicio. Ademas, una arquitectura madura minimiza la generacion dinamica de codigo y prefiere el intercambio estructurado de datos (JSON) sobre construir cadenas de codigo.

Capítulo 9

Estados persistentes: `pipeline_state.py`

La version 0.8 introdujo una maquina de estados persistente. Cada carga recibe un `package_token` unico y genera un archivo JSON en `pipeline_states/<package_token>.json`. La secuencia de estados es:

```
1 saved
2 vision_verified
3 package_created
4 synthesized
5 canonicalized
6 drive_archived
7 fts_indexed
8 embedded
9 database_synced
10 searchable
```

9.1. Intencion de la maquina de estados

Cada etapa se marca *solo* cuando concluye satisfactoriamente. Si una etapa falla, el estado conserva la ultima etapa completada, la etapa que fallo, el mensaje de error, el `package_id` y los archivos ya generados. Esto permite reanudar mediante `POST /resume/{package_token}` sin repetir la extraccion visual ni la sintesis.

Intencion del codigo

Un error de definicion ilustra el valor del disenno: la logica de `api_v08.py` ya conocia la etapa `drive_archived`, pero `pipeline_state.py` conservaba la secuencia anterior. El estado usado por el API no estaba declarado en la lista central. La leccion es que la lista de estados debe ser una *unica fuente de verdad*: un pipeline reanudable exige que productor y consumidor de estados compartan exactamente el mismo vocabulario.

Capítulo 10

Archivado idempotente: `drive_archive.py`

Este modulo independiente recibe un paquete y lo archiva en Google Drive. Su diseno enfatiza la idempotencia: la etapa `drive_archived` se marca *solo despues* de que cada copia termine correctamente, usando nombres reproducibles.

Intencion del codigo

Las integraciones externas deben disenarse pensando en reintentos. Si el archivado se interrumpe, un segundo intento con nombres reproducibles no crea duplicados ni deja el estado marcado prematuramente. La marca `drive_archived` es una promesa verificable, no un optimismo.

Un detalle de configuracion resulto critico: la primera version uso el cliente OAuth publico compartido de rclone, que devolvio un error 403 por cuota excedida. La solucion fue crear un cliente OAuth propio tipo Desktop. La leccion: una integracion de produccion debe usar sus propias credenciales y cuota.

Capítulo 11

Helper de inferencia: ollama_pdf_researcher

Para desacoplar el sandbox del motor de inferencia se introdujo un helper cuya unica tarea es recibir un *prompt* por entrada estandar, llamar a Ollama en 127.0.0.1:11434/api/generate y devolver exclusivamente el JSON producido por Nemotron.

```
1 payload = {  
2     "model": "nemotron-3-nano:30b",  
3     "prompt": prompt,  
4     "stream": False,  
5     "format": "json",  
6     "options": {  
7         "temperature": 0,  
8         "num_ctx": 16384,  
9         "num_predict": 2048  
10    }  
11 }
```

11.1. Analisis clave por clave

"model": "nemotron-3-nano:30b". Selecciona el modelo de sintesis servido localmente por Ollama.

"stream": False. Desactiva la respuesta por fragmentos. Para consumir la salida como un unico objeto JSON es mas simple recibirla completa que reensamblar tokens.

"format": "json". Instruye a Ollama para forzar salida JSON valida, un contrato a nivel de runtime que reduce (aunque no elimina) los fallos de formato.

"temperature": 0. Determinismo maximo. En un pipeline de extraccion no se busca creatividad, sino reproducibilidad: la misma evidencia debe producir la misma sintesis.

"num_ctx": 16384. Limita la ventana de contexto a 16k tokens. Acotar el contexto evita reservas excesivas de memoria en la GPU.

"num_predict": 2048. Limita la longitud de la salida, protegiendo tiempos de respuesta y memoria.

Intencion del codigo

Este helper conserva la inferencia en la interfaz loopback del Spark, desacopla el sandbox del motor, simplifica la trazabilidad y permite probar Nemotron de forma independiente. Es un ejemplo del principio de *agentes pequenos con responsabilidad unica* que atraviesa toda la arquitectura.

Capítulo 12

La capa determinista: sanitizacion de sintesis

El descubrimiento arquitectonico mas importante del proyecto es que la robustez no debe delegarse por completo al modelo. Un LLM puede producir una excelente interpretacion semantica y, a la vez, clasificar el nombre de una organizacion como si fuera una URL. La solucion no es exigir perfeccion probabilistica, sino interponer una capa determinista entre el modelo y la persistencia.

```
1 parsed = sanitize_synthesis(  
2     extract_json(result.stdout)  
3 )
```

12.1. Analisis

`extract_json(result.stdout)`. Extrae el objeto JSON desde la salida cruda del modelo. Esto conecta con un error decisivo del proyecto: en un momento el codigo esperaba `extract_json(result.stdout)` pero recibia la cadena `completed\n`. El `returncode=0` probaba que el proceso termino, no que `stdout` contuviera la respuesta del modelo. El wrapper agentic estaba tratando la respuesta como estado interno, no como salida de consola.

`sanitize_synthesis(...)`. Aplica reglas deterministas: valida tipos, corrige clasificaciones erroneas (por ejemplo, una organizacion mal marcada como URL) y garantiza que solo un objeto valido llegue a la persistencia.

Principio de diseno

Ejecucion, transporte y semantica son tres conceptos distintos. Que un proceso termine (ejecucion) no dice donde dejo la respuesta (transporte) ni si esa respuesta cumple el contrato (semantica). Un pipeline maduro verifica los tres niveles por separado.

12.2. Extraccion de tokens con Python inline

En el flujo de consulta, Python actua como filtro de una tuberia de shell para extraer un token de una respuesta JSON:

```
1 import sys, json  
2 print(json.load(sys.stdin)["ask_token"])
```

`json.load(sys.stdin)` lee y parsea el JSON que `curl` envió por la tubería hacia la entrada estándar, transformándolo en un diccionario. `["ask_token"]` selecciona el campo. `print` lo emite a `stdout`, donde el shell lo captura en una variable. Este patrón une la robustez del parseo JSON de Python con la comodidad de las tuberías de shell.

Capítulo 13

Síntesis: los principios de ingeniería del código Python

Recorrido todo el código Python del proyecto, emergen principios consistentes que explican la *intención* global más allá de cada instrucción.

1. **Determinismo alrededor de la probabilidad.** El código Python clásico envuelve al LLM: normaliza entradas, valida esquemas, sanitiza salidas. La IA se reserva para lo ambiguo; lo verificable se resuelve con código.
2. **Contratos de salida explícitos.** Ningún modelo escribe directamente en la base. Cada salida pasa por JSON Schema, validación y sanitización.
3. **Estado persistente y reanudable.** La máquina de estados por token convierte cualquier fallo en un problema local recuperable, no en la pérdida de toda la ejecución.
4. **Verificación de artefactos, no solo de terminación.** Un `returncode=0` o una función que retorna sin excepción no bastan; el proyecto verifica que el artefacto exista y cumpla su contrato.
5. **Validación sintáctica antes del despliegue.** `python3 -m py_compile` precede a cada reinicio de servicio.
6. **Inferencia local y desacoplada.** Toda la IA corre en loopback dentro del Spark; helpers de responsabilidad única separan el motor de inferencia del resto del pipeline.
7. **Idempotencia en integraciones externas.** Google Drive y otras integraciones se diseñan para tolerar reintentos sin duplicar ni marcar estados prematuramente.

Intención del código

El cambio conceptual más importante del proyecto no fue sustituir una función ni ajustar un parámetro. Fue dejar de pensar en un agente que hace todo; comenzar a pensar en una infraestructura de conocimiento compuesta por agentes pequeños, procesadores deterministas, contratos explícitos y estados persistentes. La inteligencia del sistema emerge de la coordinación entre estas piezas de código Python, no de exigir que un único agente sea simultáneamente observador, extractor, archivista, sintetizador, buscador y razonador.